AI ASSISTANT CODING

ASSIGNMENT - 18.4

Name: Aman Sarkar

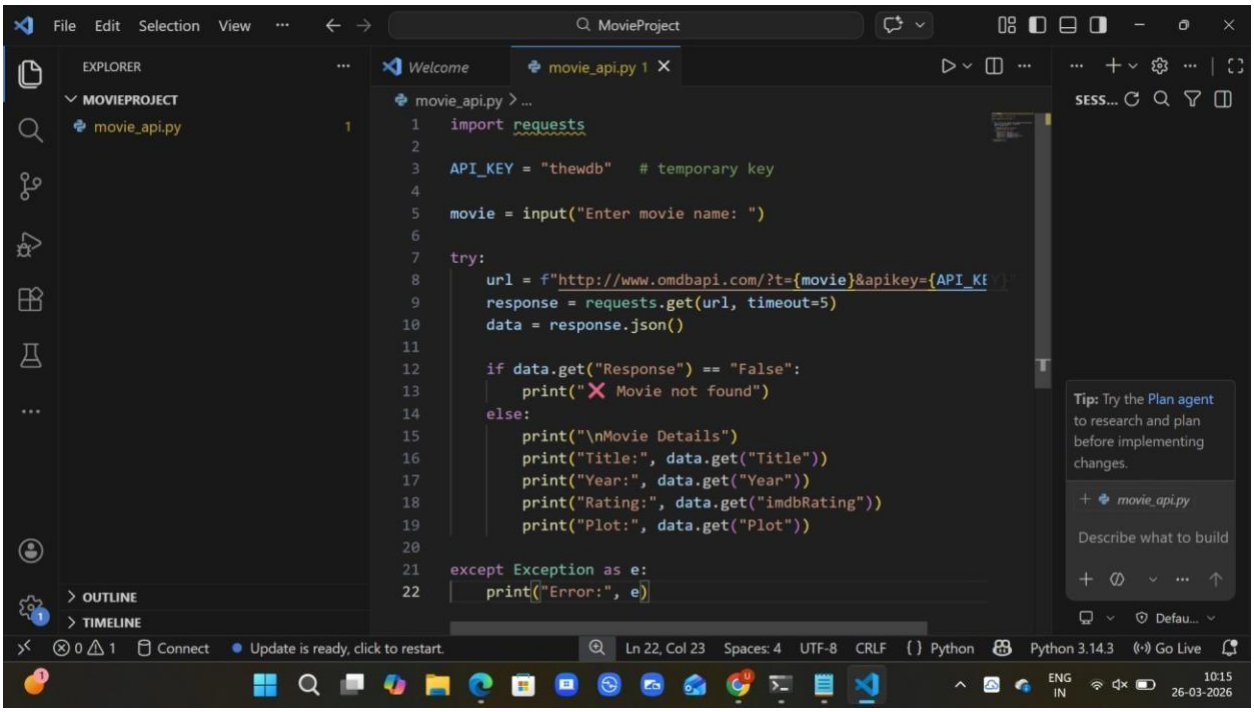
Ht.no: 2303A51273

Batch: 05

Task 1: Movie Database API Integration

Prompt:Generate a Python script to fetch movie details using OMDB API. The program should accept movie title as input and display title, year, rating, and plot. Include proper error handling for invalid movie names, API errors, and network issues.

Code:



Output:

```
PS C:\Users\uppus\Downloads\MovieProject> python movie_api.py
Enter movie name: Avatar

Movie Details
Title: Avatar
Year: 2009
Rating: 7.9
Plot: A paraplegic Marine dispatched to the moon Pandora on a unique missio
n becomes torn between following his orders and protecting the world he fee
ls is his home.
PS C:\Users\uppus\Downloads\MovieProject>
```

Explanation:

-->The program uses requests library to connect to OMDB API.

-->It takes movie name as user input.

-->The API response is converted into JSON format.

-->Required details like title, year, rating, and plot are displayed.

-->Error handling is implemented for invalid movie names and network issues.

Task 2: Public Transport Schedule API Integration

Prompt: Generate a Python script to fetch public transport arrival timings using an API.

The program should accept a stop ID as input and display the next 3 arrivals.

Include error handling for invalid stop ID, server issues, and timeout errors.

Also implement retry logic for failed API calls.

Code:

```python
import requests

stop_id = input("Enter Stop ID (example: 490008660N): ")

url = f"https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"

for attempt in range(2):
    try:
        print("Fetching data...")

        response = requests.get(url, timeout=10)  # increase

        if response.status_code != 200:
            print("❌ Invalid Stop ID or Server Error")
            break

        data = response.json()

        if not data:
            print("No arrivals found")
            break

        print("\n🚌 Next 3 Arrivals")
```

```python
        break

    except requests.exceptions.Timeout:
        print("⏱ Timeout... Retrying")
        time.sleep(2)

    except requests.exceptions.ConnectionError:
        print("🌐 Network error. Check internet")
        break

    except Exception as e:
        print("Error:", e)
        break
```

Output:

```
Enter Stop ID (example: 490008660N): 490008660N
Fetching data...

🚌 Next 3 Arrivals
🚌 Next 3 Arrivals
----------------------
214 - 2026-03-26T05:02:24Z
88 - 2026-03-26T05:16:28Z
```

Explanation:

-->The program uses the requests library to connect to a transport API and fetch arrival data.

-->It takes stop ID as user input to retrieve specific station details.

-->The API response is converted into JSON format and processed.

-->It displays the next 3 arrival timings in a readable format.

Task 3: Stock Market Data API Integration

Prompt: Generate a Python script to fetch stock market data using a financial API. The program should accept a stock symbol as input and display current price, day high, and day low. Include error handling for invalid symbols, API rate limits, and missing data in the API response.

Code:

```python
import requests

# 🔑 Put your API key here
API_KEY = "demo"   # you can use "demo" for testing

symbol = input("Enter stock symbol (example: IBM): ")

try:
    url = f"https://www.alphavantage.co/query?function=GLOBA

    response = requests.get(url, timeout=5)
    data = response.json()

    stock = data.get("Global Quote", {})

    # ❌ Invalid symbol
    if not stock:
        print("❌ Invalid stock symbol or no data found")

    else:
        print("\n📈 Stock Details")
        print("----------------------")
        print("Price     :", stock.get("05. price", "N/A"))
```

```python
    else:
        print("\n📈 Stock Details")
        print("----------------------")
        print("Price     :", stock.get("05. price", "N/A"))
        print("Day High  :", stock.get("03. high", "N/A"))
        print("Day Low   :", stock.get("04. low", "N/A"))

# ⏱ Timeout
except requests.exceptions.Timeout:
    print("⏱ Request timed out")

# 🌐 Network error
except requests.exceptions.ConnectionError:
    print("🌐 Network error")

# ⚠ Other errors
except Exception as e:
    print("Error:", e)
```

Output:

```
PS C:\Users\uppus\Downloads\MovieProject> python stock_api.py
Enter stock symbol (example: IBM): IBM

📈 Stock Details
---------------------
Price     : 241.3900
Day High  : 246.1900
Day Low   : 238.0000
PS C:\Users\uppus\Downloads\MovieProject>
```

Explaantion:

-->The program uses the requests library to fetch stock data from a financial API.

-->It takes stock symbol as input from the user (e.g., IBM).

-->The API response is converted into JSON format and processed safely.

-->It displays important details like current price, day high, and day low.

-->Error handling is implemented for invalid symbols, network issues, and missing JSON fields.

Task 4: **Geolocation API (IP Address Lookup)**

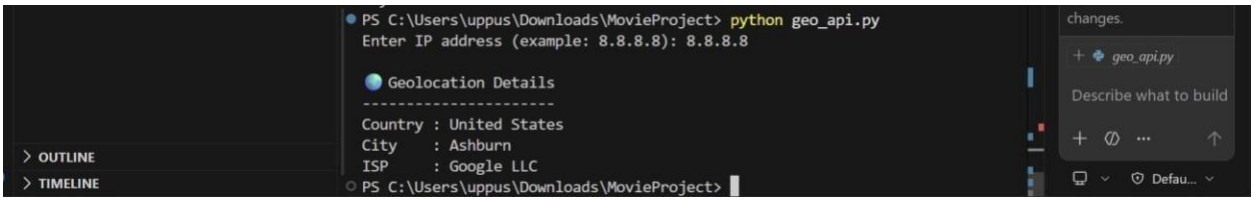
Prompt: Generate a Python script to fetch geolocation details using an IP address. The program should accept an IP address as input and display country, city, and ISP. Include validation for IP format and error handling for invalid input, API failure, and JSON decoding errors.

Code:

```python
import requests
import re

# 📥 Take IP input
ip = input("Enter IP address (example: 8.8.8.8): ")

# 🔍 Validate IP format
pattern = r"^\d{1,3}(\.\d{1,3}){3}$"

if not re.match(pattern, ip):
    print("❌ Invalid IP format")
else:
    try:
        url = f"http://ip-api.com/json/{ip}"
        response = requests.get(url, timeout=5)

        # Convert to JSON
        data = response.json()

        # Check API status
        if data.get("status") != "success":
            print("❌ Failed to fetch location")
        else:
```

```python
        else:
            print("\n🌍 Geolocation Details")
            print("---------------------")
            print("Country :", data.get("country", "N/A"))
            print("City    :", data.get("city", "N/A"))
            print("ISP     :", data.get("isp", "N/A"))

    except requests.exceptions.Timeout:
        print("⏱ Request timed out")

    except requests.exceptions.ConnectionError:
        print("🌐 Network error")

    except ValueError:
        print("⚠ JSON decoding error")

    except Exception as e:
        print("Error:", e)
```

Output:



Explanation:

-->The program accepts an IP address as user input.

-->It validates the IP format using regular expressions (regex).

-->It uses the requests library to connect to a geolocation API.

-->The API response is converted into JSON format and parsed.

-->Error handling is implemented for invalid IP, network issues, API failure, and JSON errors.

Task 5: Real-Time Application – Payment Gateway Simulation API

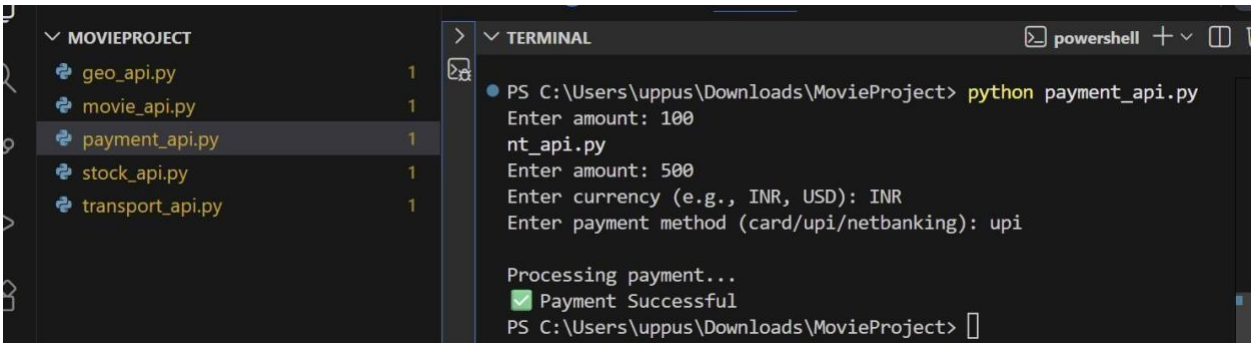
Prompt: Generate a Python script to simulate payment processing using a sandbox API. The program should send a POST request with amount, currency, and payment method. Include input validation and handle errors such as invalid payment details, HTTP status errors (400, 401, 500), and network issues. Also log failed transactions into a file.

Code:

```python
import requests

# 📥 User input
amount = input("Enter amount: ")
currency = input("Enter currency (e.g., INR, USD): ")
method = input("Enter payment method (card/upi/netbanking): ")

# 🔍 Basic validation
if not amount.isdigit():
    print("❌ Invalid amount")
else:
    try:
        url = "https://httpbin.org/post"

        payload = {
            "amount": amount,
            "currency": currency,
            "method": method
        }

        print("\nProcessing payment...")

        response = requests.post(url, json=payload, timeout=5)
```

```python
        elif response.status_code == 500:
            print("❌ Server Error")
        else:
            print("❌ Payment Failed with status:", response.status_code)

    # ⏱ Timeout
    except requests.exceptions.Timeout:
        print("⏱ Request timed out")

    # 🌐 Network error
    except requests.exceptions.ConnectionError:
        print("🌐 Network error")

    # ❗ Other errors + logging
    except Exception as e:
        print("Error:", e)

        # 📝 Log error to file
        with open("error_log.txt", "a") as f:
            f.write(str(e) + "\n")
```

Output:



Explanation:

-->The program collects payment details (amount, currency, method) from the user.

-->It sends a POST request to a sandbox API to simulate payment processing.

-->Input validation is performed to ensure the amount is valid before processing.

-->It handles errors such as HTTP status errors (400, 401, 500), timeout, and network issues.

-->Failed transactions are logged into a file for future reference and debugging.